RELATÓRIO TÉCNICO – SIMULADOR DE ATENDIMENTO BANCÁRIO Disciplina: INF 116 – Programação para Ciência de Dados I **Estudante:** Luan Leal Serrão Ferraz - 127043

1. Estrutura de Dados Escolhida Para gerenciar a fila de clientes do Banco Boa Sorte S.A., a opção mais prática foi usar uma lista padrão do Python contendo outras sub listas internas. Na prática, isso funciona como uma matriz dinâmica. Cada pessoa que entra na fila é representada por uma estrutura menor com três dados indexados: o número da senha (inteiro), o tipo do atendimento (caractere "C" ou "P") e o tempo de espera acumulado, que começa sempre em zero. A escolha ganhou preferência pela flexibilidade das listas nativas do Python. Funções prontas como `append` resolvem a inserção ao final da fila sem complicação, enquanto o método `pop` permite retirar dados de qualquer ponto da memória, eliminando a necessidade de fixar um tamanho máximo para o vetor logo na abertura do programa.

2. Implementação do Algoritmo de Chamada e Regras de Prioridade A lógica do menu de chamada funciona através de buscas lineares sequenciais, utilizando estruturas condicionais de checagem. Assim que a opção de chamada é acionada, o programa faz um rastreamento do início ao fim da lista principal para avaliar três regras em ordem de importância. Primeiro, o código busca algum cliente comum com tempo de espera igual ou superior a 8 unidades. Se encontrar alguém nessa condição, o processo seleciona o registro, interrompe o laço com um `break` e realiza o atendimento. Se ninguém cumprir esse teto de tolerância, o algoritmo inicia um segundo laço focado apenas na marcação preferencial "P". Não havendo prioritários, uma última varredura puxa o primeiro cliente comum "C" disponível na fila. Essa sequência de laços `for` garante que as exigências do enunciado funcionem sem conflitos, deixando a própria ordem de chegada tratar os casos de desempate natural.

3. Mecanismo de Avanço de Tempo e Tratamento de Desistências O comando de atualização do tempo executa duas tarefas distintas na fila e exige atenção redobrada com os índices das posições. No primeiro momento, um laço varre a lista incrementando em uma unidade o contador de tempo de cada sublista. Logo na sequência, o sistema precisa localizar e remover quem atingiu 10 tempos de espera para processar a desistência automática. Para fazer essa limpeza de forma segura, a saída foi aplicar um laço `while` que roda em ordem reversa, indo do último índice até o zero. Essa abordagem de trás para frente é crucial quando se manipula listas dinâmicas. Se a remoção rodasse do início para o fim, as posições dos elementos restantes mudariam de lugar durante a execução do laço, fazendo o Python pular checagens ou quebrar o programa por erro de índice fora de alcance (`IndexError`). O formato reverso garante que mesmo se múltiplos clientes desistirem juntos, a lista seja limpa corretamente.

4. Controle Estatístico e Validações do Sistema O controle das métricas da agência é alimentado por variáveis globais simples que funcionam como contadores acumuladores. Elas guardam o volume de senhas, atendimentos, saídas por desistência e a soma do tempo de quem passou pelos caixas. Pensando na estabilidade do fluxo, o código recebeu travas de segurança antes de operações críticas. No bloco das estatísticas, o cálculo da média do tempo de espera fica protegido por uma condicional `if` que valida se o número de

atendidos é maior que zero. Se o banco abrir e o painel for consultado sem atendimentos prévios, o sistema exibe um aviso em texto e cancela a operação matemática, evitando o erro de divisão por zero (`ZeroDivisionError`) no terminal. Validações semelhantes impedem que o usuário tente listar a fila ou chamar pessoas com o banco vazio, blindando o menu contra interrupções abruptas.

5. Principais Dificuldades Encontradas A principal barreira prática durante o desenvolvimento do projeto ocorreu no controle do escopo e atualização das variáveis globais acumuladoras de estatísticas (como total de atendidos e soma dos tempos) dentro da estrutura principal do menu. Inicialmente, o sistema apresentou inconsistências de contagem porque os valores eram reiniciados incorretamente ou não atualizavam de maneira síncrona a cada remoção por chamada. Além disso, a formatação visual da tabela no terminal usando tabulações (`\t`) gerou desalinhamentos visuais difíceis de ajustar quando as senhas acumulavam dígitos maiores, exigindo testes repetidos de impressão para garantir que a leitura dos dados ficasse limpa e organizada para o usuário final.